Chapter 1: Introduction

1.1 Background

Marriage registration is an important civil-record process because it creates a formal record containing information about the two parties, the marriage data, registration date and place of marriage. A digital registration system can provide a structured interface for collection this information, storing supporting documents, routing applications for administrative review and making approve records searchable.

The submitted project implements this concept as an e-governance web application. The application separates the public/user-facing registration workflow from the administrative review workflow. It also introduced automated preliminary checks intended to identify unclear photographs, missing faces and potentially invalid identity documents before and application administrative reviews.

1.2 Problem Statement

A manual or loosely structured marriage-registration workflow can involve repetitive data entry, physical document handling, difficulty tracking application status and delays in verification. There is also a needed to check whether uploaded identity documents and photographs contain sufficient information for further reviews.

The project therefore addresses the needed for a centralized web application that can collect bride and groom information, receive digital documents, perform automated preliminary validation, store records in a relational database, provide an administrative review interface and expose a verification mechanism for registered records.

1.3 Objectives

- Develop a web-bases marriage registration application using Django.
- Provide registration and authentication facilities for application users.
- Collect structured bride, groom and marriage information.
- Accept digital photographs and NID document images.
- Provide an administrative dashboard for monitoring and reviewing requests.
- Notify applicants of approval or rejection through email.
- Generate and display a marriage certificate for approved records.

1.4 Scope

In scope	Description
User accounts	Registration login and logout using Django authentication
Marriage application	Bride, groom, marriage date, registration date and marriage location
Documents upload	Bride/Groom photographs and NID documents images.
OCR validation	Text extraction and keywords/NID checks on uploaded NID images.
Image validation	Image clarity and analysis plus face and eye detection
Administration	Request listing, detail viewing ,approval and rejection.
Notification	Email message for approval/rejection.
Certificate	Creation of a marriage record and certificate number After approval

Verification	Search for a registered marriage by NID number.
Information pages	Help, terms ,privacy data protection disclaimer and contact pages.

1.5 Limitations

- The OCR and computer-vision checks are preliminary validation mechanism, and should not be treated as legal identity verification by themselves.
- The current information uses fixes OCR keywords and a fixed image-quality threshold, so recognition performance can vary with document layout, lighting, resolution and image quality.
- The current sources code contains development configuration such as DEBUG = True and locally configures credentials; production deployment requires security hardening.
- The administrative login views authenticate as Django user but doesn't itself enforce a staff/superuser check.

1.6 Significance

The project demonstrates a practical e-governance workflow in which web forms, relational database storage, documents processing and computer vision are integrated into a single application. Form an academic perspective it illustrates how Django can coordinate authentication request processing media uploads, ORM operations, email services and machine-assisted validation.

Chapter 2: Literature and Technology Review

2.1 E-governance Systems

E-governance applications use information technology to provide public or administrative services digitally. Common characteristics include centralized records, role-based access online submission, status tracking, searchable information and automated notifications. The proposed project follows these characteristics by combining citizen-facing form with an administrative review process.

2.2 Django

Django is a python web framework based on the model-Template-View architecture. In this project, models define persistent data structures, views implement request-processing logic, templates render the user interface, URL patterns map browser request to views, and Django's authentication framework manages user credentials and sessions.

2.3 PostgresSQL

PostgresSQL is use as the configures relational databases. IT is appropriate for structures recordsbecause the project contains related entities such as bride, groom, marriage request and marriage record. Django's ORM provides the application-level interface for creating, retrieving, filtering and updating these records.

2.5 Tesseract OCR

Tesseract OCR is used through the pytesseract python package to extract text from uploaded indentity-document images. The application normalizes extracted text and checks for a set expected terms and the NID values entered by the applicant.

2.5 Open CV

OpenCV is used for image processing and Haar-cascade detection. The implementation converts photographs to grayscale, calculates Laplacian variance as a focus/clarity indicator, detects faces using a frontal-face Haar cascade and checks for eyes inside detected face region.

2.5 Front-End Technologies

Technology	Description
HTML	Defines pages structures, forms, navigations and certificate layout.
CSS	Provides pages styling, responsive, layouts cards form and certificate presentations.
JS	Preforms client-side marriage-form validation and interacts with from fields.
Font Awesome	Provides interface icons used throughout the templates

Chapter 3: System Analysis

3.1 Existing-System Issues

- Dependence on manual collection and checking of application information.
- Repeated physical handling of identity documents and photographs.
- Difficulty maintaining a centralized searchable marriage record.
- Limited automation in preliminary documents and image checks.
- Manual communication of application decisions.

3.2 Proposed systems

The proposed system provides an end-to-end digital workflow. A user creates an account, logs in and submits bride/groom and marriage information together with photographs and NID images. The server performs OCR and image validation before saving the application. An administrator can inspect submitted information, approve or reject the request, and the system can notify the groom by email. Approval results in a MarriageRecorde entry with a certificate number. A separate verification page can search approved marriage records by NID number.

3.3 User Roles

Role	Main Responsibilities
Registered User	Create account, log in, submit marriage application and access system information.
Administrator	View requests, inspect details, approve/reject applications, monitor dashboard and manage records.

3.4 Functional Requirements

ID	Requirement
FR-01	The system shall allow a user to register with a username, email and password.
FR-02	The system shall authenticate users using username and password.
FR-03	The system shall allow authenticated users to submit marriage information.
FR-04	The system shall accept bride and groom photographs and NID images.
FR-05	The system shall run OCR processing on uploaded NID images.
FR-06	The system shall perform image clarity validation.
FR-07	The system shall detect a face and eyes in bride/groom photographs.
FR-08	The system shall store successful applications in the database.
FR-09	The administrator shall be able to list and inspect requests.
FR-10	The administrator shall be able to approve or reject a request.
FR-11	The system shall create a marriage record after approval.

FR-12	The system shall send approval/rejection notification emails when an email address is available.
FR-13	The system shall display a marriage certificate for an approved record.
FR-14	The system shall allow marriage-record verification by NID number.

3.5 Non-Functional Requirements

Category	Requirement
Usability	Forms and pages should be understandable and provide clear validation messages.
Performance	Database operations should use indexed relational queries and image processing should be kept within reasonable request time.
Security	Authentication, CSRF protection, input validation and secure secret management should be used.
Reliability	Invalid submissions should be rejected before database creation where possible.
Maintainability	Django models, views, templates, CSS and JavaScript should remain separated by responsibility.
Scalability	PostgreSQL provides a foundation for larger record volumes and concurrent use.
Availability	The deployed service should provide reliable access to registration and verification functions.

3.6 Feasibility Study

Type	Assessment
Technical	Feasible using Django, PostgreSQL, OpenCV and Tesseract OCR. All core components are supported by Python.
Operational	The workflow is suitable for a user/admin model and reduces repeated manual processing.
Economic	The project relies primarily on open-source technologies, reducing software licensing costs.
Schedule	The modular architecture supports incremental development and testing.
Security	Technically feasible, but production deployment requires stronger secret handling, access control and secure media management.

Chapter 4: System Design

4.1 High-Level Architecture

The application follows a layered web architecture. The browser provides the presentation layer through HTML/CSS/JavaScript. Django receives HTTP requests and executes application logic. The ORM communicates with PostgreSQL. Uploaded NID images are passed to Tesseract OCR, while photographs are passed through OpenCV processing. The application also uses Django's email facilities for decision notifications.

Layer	Components	Responsibilities
Presentation	HTML, CSS, JavaScript	Forms, pages, validation, navigation and certificate display.
Application	Django views, authentication, URL routing	Request processing, business workflow, validation and email actions.
Processing	pytesseract, OpenCV, PIL/Pillow, NumPy	OCR, image decoding, image clarity and face/eye detection.
Data	Django ORM, PostgreSQL	Persistent storage and relational queries.
Media	Django MEDIA_ROOT / uploaded files	Storage of photographs and identity-document images.

4.2 System Flow

2 User opens the application and creates an account.
3 User logs in with Django authentication.
4 User opens the marriage-registration form.
5 User enters bride, groom and marriage details.
6 User uploads photographs and NID images.
7 Client-side JavaScript checks required fields and files.
8 Django receives the multipart form submission.
9 Tesseract extracts text from bride and groom NID images.
10 OCR text is normalized and checked for configured keywords and entered NID values.
11 OpenCV evaluates image clarity and detects faces and eyes.
12 If validation succeeds, Bride, Groom and MarrigaeRequest records are created.
13 Administrator views the request and its details.
14 Administrator approves or rejects the application.
15 Approval creates a MarriageRecorde and certificate number; rejection changes the request status.
16 An email notification is attempted.
17 Approved records can be displayed as a marriage certificate and searched using the verification page.

4.3 Use Case Description

Actor	Use Case	Description
User	Register	Creates a Django user account.
User	Login	Authenticates using username/password.
User	Submit Marriage Request	Enters couple and marriage information and uploads documents.
System	Validate Documents	Runs OCR and image validation.
Administrator	View Requests	Lists submitted marriage requests.
Administrator	View Details	Inspects bride/groom information and uploaded images.
Administrator	Approve	Approves request and creates certificate record.
Administrator	Reject	Rejects request and records rejection status.
System	Send Email	Sends an application-decision notification.
Verifier	Verify Marriage	Searches records by bride/groom NID.
User/Administrator	View Certificate	Displays an approved marriage certificate.

4.4 Data Flow Description

The primary data flow begins at the registration form. Applicant input and uploaded files are transferred to the Django view. Text documents are sent to OCR processing, while photographs are decoded and analyzed by OpenCV. Only after the implemented validation conditions are satisfied does the application create the Bride, Groom and MarrigaeRequest database records. Administrative actions then update the request and, on approval, create a MarriageRecorde.

4.5 Database Relationship Design

The database consists of four project-specific entities. Bride and Groom store personal and identity information. MarrigaeRequest links one bride and one groom and stores the marriage request details and status. MarriageRecorde has a one-to-one relationship with MarrigaeRequest and stores the certificate number and issue date.

Entity	Key / Relationship	Purpose
Bride	Primary key: id	Stores bride personal details, NID number, photograph and NID image.
Groom	Primary key: id	Stores groom personal details, NID number, photograph and NID image.

MarrigaeRequest	FK: groom, FK: bride	Stores marriage/registration dates, place and request status.
MarriageRecorde	OneToOne: marriageRequest	Stores certificate number and issue date for approved records.
auth_user	Django built-in user table	Stores application authentication accounts.

4.6 Database Dictionary

Table / Model	Important Fields	Description
Bride	brideName, brideDOB, brideEmail, brideFatherName, brideMotherName, brideAddress, brideNidNo, brideImage, brideNidImage, is_valid, create_at	Bride identity and application information.
Groom	groomName, groomDOB, groomEmail, groomFatherName, groomMotherName, groomAddress, groomNidNo, groomImage, groomNidImage, is_document_valid, create_at	Groom identity and application information.
MarrigaeRequest	groom, bride, status, marriageDate, registrationDate, place_of_marriage	Marriage registration request.
MarriageRecorde	marriageRequest, certificateNo, issuedDate	Approved marriage record/certificate metadata.

Chapter 5: Implementation

5.1 Project Structure

Path / Component	Purpose
Egovenance/app/models.py	Defines Bride, Groom, MarrigaeRequest and MarriageRecorde.
Egovenance/app/views.py	Contains registration, authentication, validation, administration, certificate and verification logic.
Egovenance/app/urls.py	Maps application URLs to Django views.
Egovenance/Egovenance/settings.py	Configures installed apps, middleware, PostgreSQL, media, static files and email.
templates/	Contains HTML templates for public, user and administrative pages.
app/static/CSS/	Contains stylesheet files for application pages.
app/static/JS/	Contains JavaScript resources, including marriage-form validation.
media/documents/	Contains uploaded image/document files in the current project.

5.2 User Registration and Authentication

The registration view reads username, email and password from the POST request and creates a Django User using User.objects.create_user(). Login uses Django's authenticate() function and login() session mechanism. Logout calls Django's logout() function and displays a success message.

5.3 Marriage Registration Module

The marriage-registration view collects two groups of personal information. Bride fields include name, date of birth, father's name, mother's name, address, email and NID number, together with a photograph and NID image. Groom fields contain the corresponding information. The marriage section collects marriage date, registration date and place of marriage.

5.4 Client-Side Validation

The marriageregister.js file checks whether required text fields are empty and whether all four required files have been selected. It displays validation messages in an error element before the form is submitted.

Implementation	observation
Client-side validation improves usability but cannot be trusted as the only validation layer. The server-side view already performs important file and processing checks; production code should also enforce every business rule on the server.	

5.5 OCR-Based NID Validation

The server sets the Tesseract executable path and uses PIL/Pillow to open the bride and groom NID images. pytesseract.image_to_string() extracts text. The extracted strings are converted to uppercase and spaces are removed. The application checks for configured terms such as NID, NATIONAL IDENTITY CARD, NEPAL and CITIZEN. It also normalizes the entered NID values and checks whether they occur in the corresponding OCR text.

Step	Implementation
1	Open uploaded bride and groom NID images using PIL.
2	Run Tesseract OCR on both images.
3	Normalize OCR text using uppercase conversion and space removal.
4	Check configured identity-document keywords.
5	Normalize entered bride/groom NID values.
6	Check whether each entered NID occurs in corresponding OCR text.
7	Use the implemented counter threshold to reject insufficient matches.

5.6 Image Clarity Validation

The application decodes uploaded photographs with NumPy and OpenCV, converts them to grayscale and computes the variance of the Laplacian. The implementation uses a threshold of 100: a score below 100 causes the corresponding image to be rejected as unclear.

Conceptually:

Clarity Score = Variance(Laplacian(Grayscale Image))

5.7 Face and Eye Detection

The system loads OpenCV's haarcascade_frontalface_default.xml and haarcascade_eye.xml. For each bride and groom photograph, detectMultiScale() is called on the grayscale image. The implementation uses scaleFactor=1.3 and minNeighbors=7. A detected face is required, and at least one detected eye inside a detected face region is also required.

Validation	Condition in Current Implementation
Face	At least one frontal face must be detected.
Eyes	At least one eye must be detected within a detected face region.
Clarity	Laplacian variance must be at least 100.
Failure	The request is returned to the registration page with an error message.

5.8 Database Persistence

After successful preliminary validation, the application creates a Bride object, a Groom object and a MarrigaeRequest object. The uploaded images are assigned to the corresponding ImageField fields. The request initially uses the model's default status and is subsequently updated by the administrative workflow.

5.9 Administrative Request Management

The request page retrieves all MarrigaeRequest objects. The detail view retrieves an individual request by its ID and displays associated bride and groom data. The dashboard calculates total users, total requests, pending requests, rejected requests and approved requests, and displays the latest five requests.

5.10 Approval Workflow

When an administrator approves a request, the view changes the request status to Approve and sets is_valid to True. It checks whether a MarriageRecorde already exists. If one does not exist, a random six-digit certificate number is generated and a new MarriageRecorde is created. The system then attempts to send an approval email containing a certificate link.

5.11 Rejection Workflow

The rejection view obtains the request, prepares a rejection email explaining that incorrect or unverifiable information/documents were provided, changes the request status to Reject and sets is_valid to False. The user can subsequently submit a new application.

5.12 Certificate Generation and Presentation

The certificate view retrieves MarriageRecorde using its marriageRequest relationship. The certificate template presents the government/office heading, certificate number, registration date, photographs, bride and groom details, marriage date, marriage place, district and province, together with an official-seal/signature area. The page includes browser print functionality.

5.13 Marriage Verification

The verification page accepts a query parameter and searches MarriageRecorde through the related bride and groom NID fields. If a record is found, the page displays a verification result containing the couple's information, certificate number, registration date and record status.

Chapter 6: User Interface and Modules

Page / Module	Purpose
Home	Landing page and navigation to major system functions.
Register	Creates a user account.
Login	Authenticates a user.
Marriage Register	Collects bride, groom and marriage data with uploads.
Request Page	Administrative list of marriage requests.
View Details	Displays complete application details and uploaded information.
Dashboard	Shows summary statistics and recent requests.
Marriage Certificate	Displays an approved marriage certificate.
Marriage Verification	Searches registered records using NID.
Help & Information	Provides system help/information.
Privacy / Data Protection	Provides information pages concerning privacy and data handling.
Terms / Disclaimer	Provides legal and usage information pages.
Contact / Report Issue	Provides support/contact functionality.

6.1 Registration Screen

The registration interface provides fields for account creation. The Django view uses the built-in User model, allowing the application to use Django's standard authentication/session mechanisms.

6.2 Marriage Application Screen

The marriage application is divided into bride information, groom information and marriage request information. Each person has a photograph and NID image upload. JavaScript provides immediate feedback for missing values/files, while the server performs the actual document and image processing.

6.3 Administrative Dashboard

The dashboard presents operational counts derived from the database. It includes total users, total requests, pending requests, rejected requests and approved requests, along with a recent request list.

6.4 Verification Interface

The verification interface is designed as a public record-search page. A verifier enters an NID number and the system searches both bride and groom NID fields. A successful search returns the associated marriage record and certificate details.

Chapter 7: Testing

7.1 Testing Strategy

Testing should cover the complete request lifecycle as well as individual validation components. The following test cases are derived from the implemented workflow and represent the minimum functional test plan for the project.

ID	Test	Expected Result	Acceptance
TC-01	Register with valid username/email/password	Account is created and user is redirected to login.	Pass criteria
TC-02	Login with valid credentials	User is authenticated and redirected to home.	Pass criteria
TC-03	Login with invalid username	Username-not-found message is shown.	Pass criteria
TC-04	Submit without bride image	Server rejects submission with upload error.	Pass criteria
TC-05	Submit without groom NID image	Server rejects submission with upload error.	Pass criteria
TC-06	NID OCR does not satisfy validation	Application returns 'Document is not valid'.	Pass criteria
TC-07	Blurry bride/groom photograph	Application rejects image when clarity score is below threshold.	Pass criteria
TC-08	Photograph without detectable face	Application rejects corresponding image.	Pass criteria
TC-09	Photograph with face but no detected eye	Application rejects corresponding image.	Pass criteria
TC-10	Valid application	Bride, Groom and MarrigaeRequest records are created.	Pass criteria
TC-11	Admin opens request details	Correct bride/groom and request information is displayed.	Pass criteria
TC-12	Admin approves request	Status becomes approved and MarriageRecorde is created.	Pass criteria
TC-13	Admin rejects request	Status becomes rejected and rejection email is attempted.	Pass criteria
TC-14	Verify existing NID	Registered marriage record is displayed.	Pass criteria
TC-15	Verify unknown NID	No-record message is displayed.	Pass criteria
TC-16	Open certificate for approved request	Certificate information is rendered and printable.	Pass criteria

7.2 Validation Test Matrix

Input Condition	Expected System Behavior
Missing required text field	Client-side validation displays an error; server should also reject invalid data.
Missing uploaded file	Server displays the relevant upload error.
Invalid NID document	OCR counter remains below threshold and submission is rejected.
Unclear photograph	Laplacian variance below 100 causes rejection.
No face detected	Submission is rejected for the relevant person.
No eye detected	Submission is rejected for the relevant person.
Valid data and images	Database records are created.

Chapter 8: Security and Privacy Considerations

8.1 Authentication

Django's authentication framework is used for account creation, credential verification and session login/logout. Passwords are handled by Django's password hashing mechanisms when create_user() is used.

8.2 CSRF and Middleware

The project enables Django's CSRF middleware and common security middleware. POST forms should continue to include CSRF tokens in production templates.

8.3 Sensitive Documents

NID images and personal information are sensitive records. Media directories should not be assumed to be publicly accessible in a production deployment. Access-control rules, protected media delivery, secure storage, retention policies and audit logging should be implemented.

8.4 Secret Management

The analyzed settings file contains development-time secret/database/email credentials. These must not be committed to source control or reproduced in a submission appendix. Production configuration should use environment variables or a secrets manager, with a non-development SECRET_KEY and DEBUG=False.

8.5 Administrative Authorization

The current administrative login view uses authenticate() but does not explicitly require is_staff or is_superuser. A production system should enforce a dedicated administrative role or staff/superuser check before granting access to request-management and dashboard views.

8.6 Data Validation

OCR and OpenCV checks should be treated as screening mechanisms. They can produce false positives or false negatives depending on image quality and document format. Final approval should remain an authorized administrative/legal process.

Chapter 10: Conclusion and Future Enhancements

10.1 Conclusion

The E-Governance Marriage Registration System demonstrates a complete web-based workflow for collecting and managing marriage-registration applications. Django provides the core application framework and authentication system, PostgreSQL provides structured relational storage, and HTML/CSS/JavaScript provide the user interface. Tesseract OCR and OpenCV add automated preliminary validation of identity documents and photographs.

The system also demonstrates an administrative lifecycle: applications are submitted, screened, stored, reviewed and either approved or rejected. Approved applications receive a marriage record and certificate number, while email notifications communicate the result. The NID-based verification module provides a separate mechanism for locating registered records.

For academic purposes, the project is a useful demonstration of integrating web development, database management, OCR, computer vision and administrative workflow. For real-world deployment, additional security, legal, privacy, identity-verification and document-integrity controls would be required.

10.2 Future Enhancements

Implement role-based access control for citizens, registrars, supervisors and system administrators.

- Add application status tracking for users.
- Add secure document storage and protected media access.
- Improve OCR through image preprocessing, field extraction and document-specific models.
- Introduce configurable document-validation rules rather than fixed keyword counters.
- Add stronger identity verification or face matching where legally permitted.
- Generate digitally controlled PDF certificates with unique verification identifiers.
- Add QR-code-based certificate verification.
- Add audit logs for every administrative action.
- Add database constraints and unique indexes for certificate numbers and identity numbers where appropriate.
- Add automated and scheduled database backups.
- Add comprehensive Django unit and integration tests.
- Add multilingual support, including Nepali language interfaces.
- Add SMS/email status notifications and user dashboards.
- Deploy with HTTPS, environment-based configuration, secure cookies and production monitoring.

References

[1]. Kafle, R. (2023). *Marriage Registration Practice in Nepal*. **Baneshwor Campus Journal of Academia, 2**(1), 58–73. - https://doi.org/10.3126/bcja.v2i1.55761

[2]. Mills, S., Lee, J. K., & Rassekh, B. M. (2019). *An introduction to the civil registration and vital statistics systems with applications in low- and middle-income countries*. **Journal of Health, Population and Nutrition, 38**, 23. - https://doi.org/10.1186/s41043-019-0177-1

[3]. Mills, S. L., Abouzahr, C., Kim, J. H., Rassekh, B. M., & Sarpong, D. (2017). *Civil registration and vital statistics (CRVS) for monitoring the Sustainable Development Goals (SDGs)*. **World Bank Group**.

[4]. Mikkelsen, L., Phillips, D. E., AbouZahr, C., Setel, P. W., de Savigny, D., Lozano, R., & Lopez, A. D. (2015). *A global assessment of civil registration and vital statistics systems: Monitoring data quality and progress*. **The Lancet, 386**(10001), 1395–1406.

Appendix A: Main URL Endpoints

URL Pattern	Name	Purpose
/	home	Home page
/register/	register	User registration
/login/	login	User login
/logout/	logout	User logout
/register_marriage_views/	marriageregister	Marriage registration form/process
/request/	request	Administrative request list
/request/viewdetails/<id>/	viewdetails	Request detail page
/request/admin_approve_views/<id>/	approve	Approve request
/request/admin_reject_view/<id>/	reject	Reject request
/marriageCertifficate/<id>/	certificate	Marriage certificate
/marriageverification/	marriageverification	NID-based marriage verification
/dashboardviews	dashboard	Administrative dashboard
/adminlogin_views/	admin	Administrative login page

Appendix B: Core Validation Algorithm

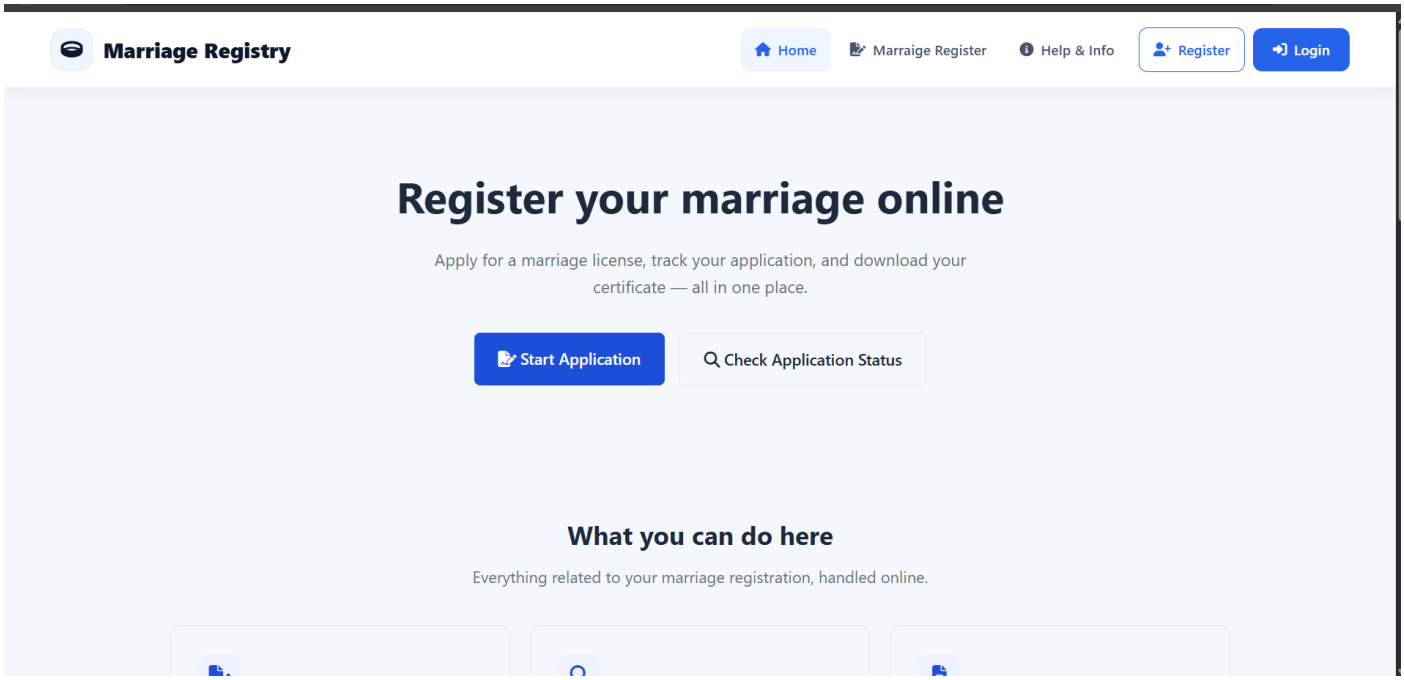
The implemented marriage-registration validation can be summarized as follows:

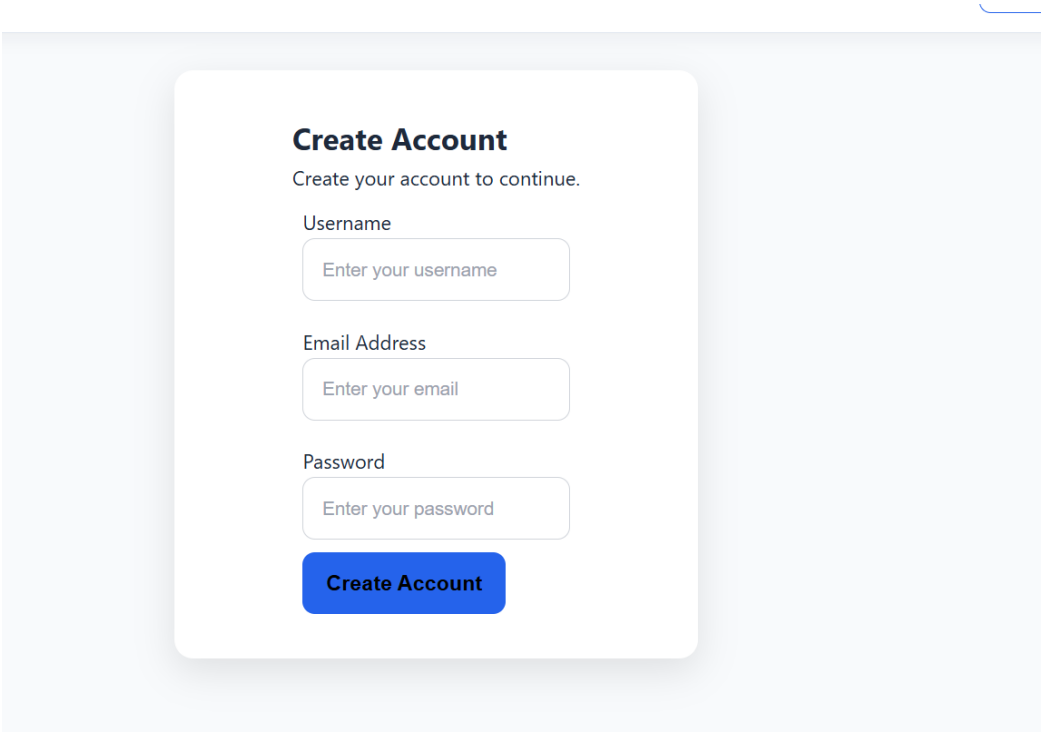
1. Receive bride/groom data and four uploaded files.
2. Reject if any required image/document is missing.
3. Run OCR on both NID images.
4. Normalize OCR text and entered NID values.
5. Count configured document keywords and matching entered NIDs.
6. Reject if the implemented OCR counter is below six.
7. Decode bride/groom photographs using OpenCV.
8. Convert photographs to grayscale.
9. Calculate Laplacian variance and reject scores below 100.

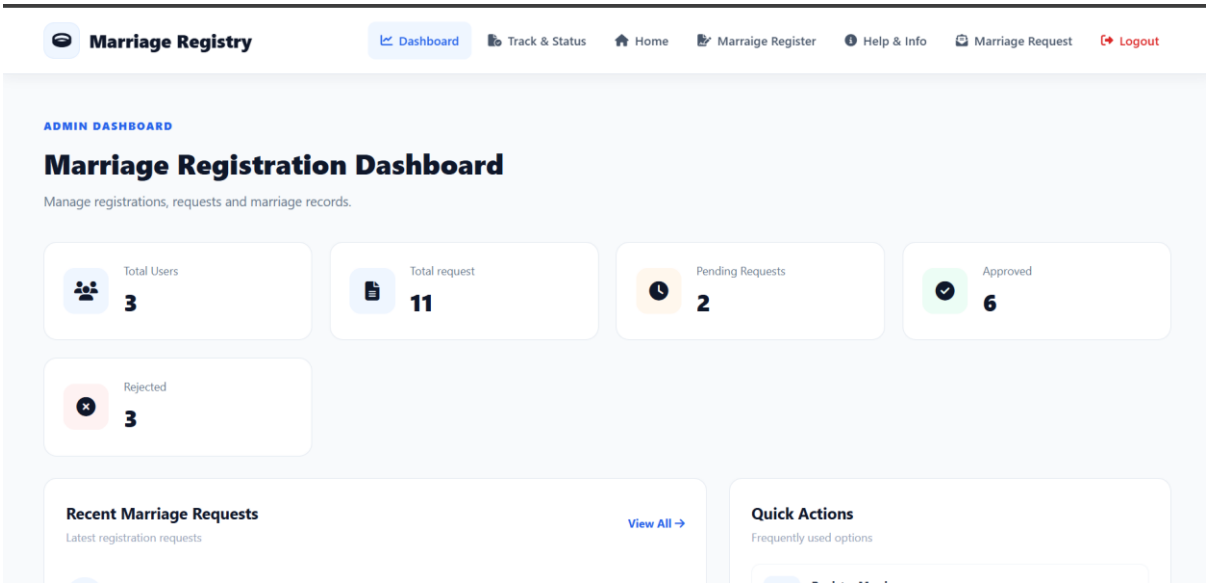
10. Detect a frontal face using Haar cascade.

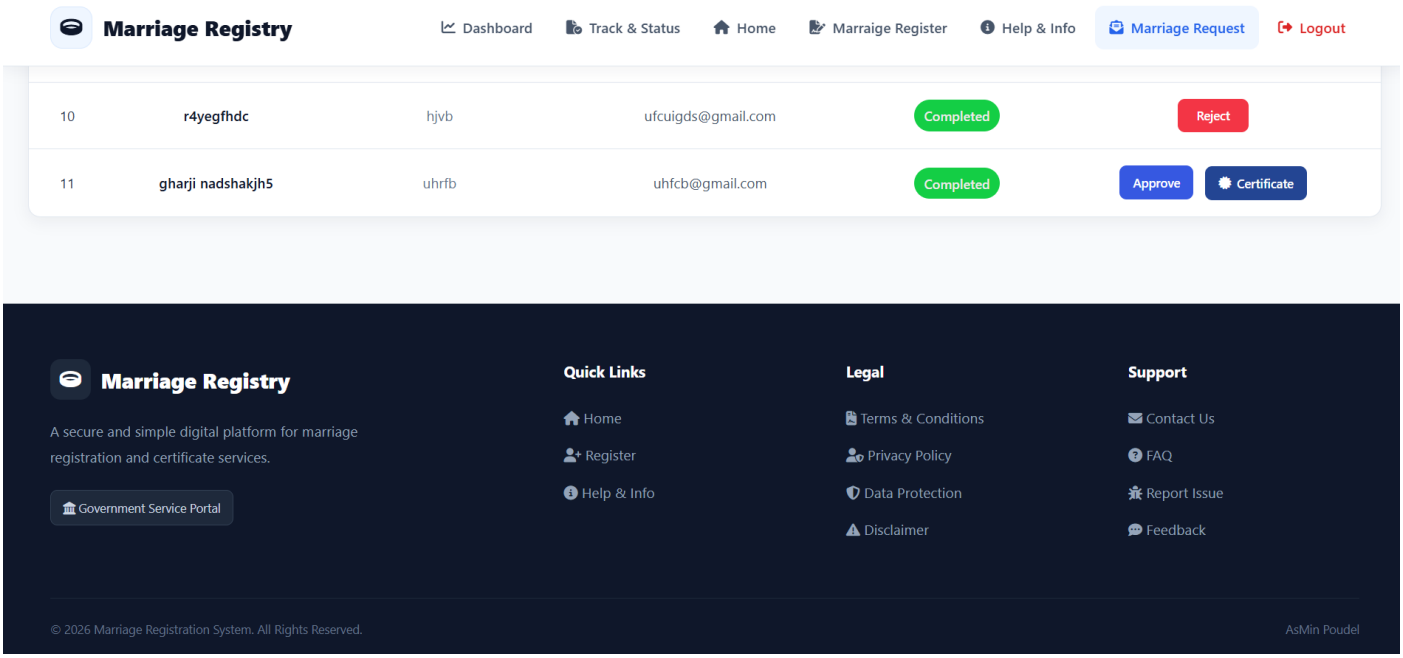
Appendix C: Project Quality Checklist

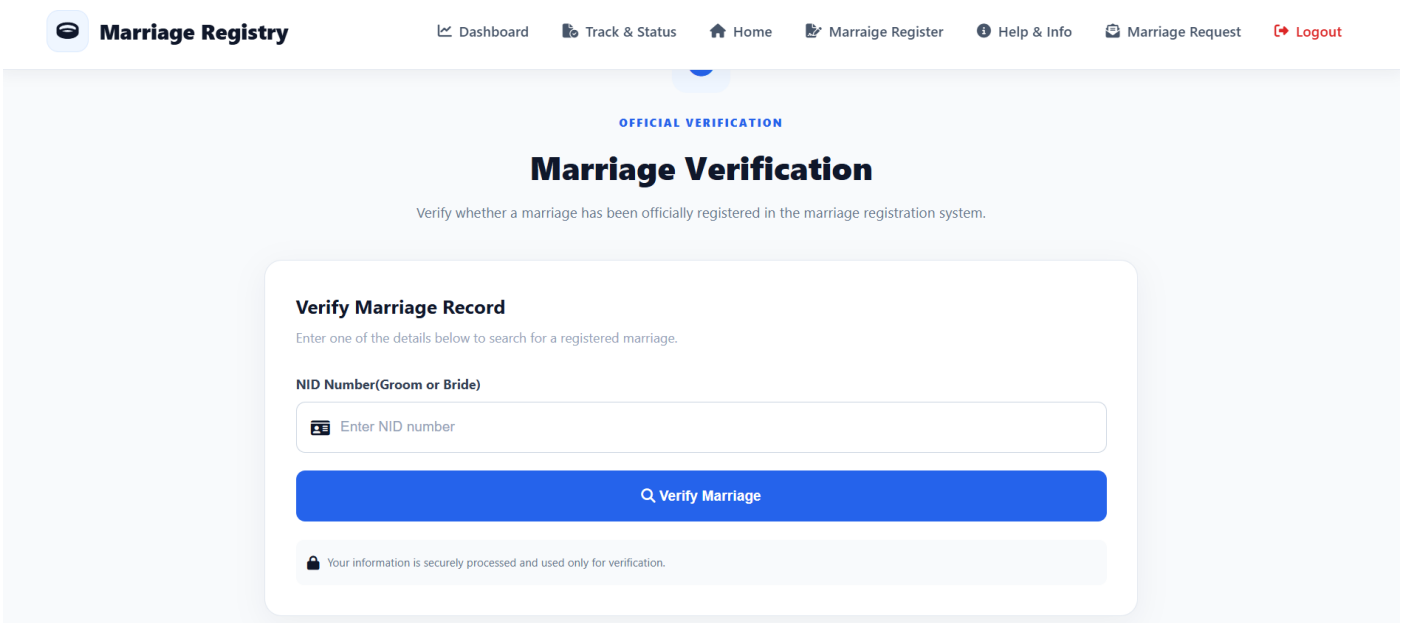
Item	Status / Recommendation
Django MVC/MVT structure	Implemented
PostgreSQL configuration	Implemented
User authentication	Implemented
Marriage request data model	Implemented
Image/document upload	Implemented
OCR validation	Implemented
OpenCV clarity check	Implemented
Face detection	Implemented
Eye detection	Implemented
Admin approval/rejection	Implemented
Email notification	Implemented / SMTP configured
Certificate record	Implemented
NID-based verification	Implemented
Production secret management	Recommended
Strict admin authorization	Recommended
Server-side date/business-rule validation	Recommended
Automated test suite	Recommended
Audit logging	Recommended
Secure production media access	Recommended
Digitally controlled certificate/PDF	Recommended

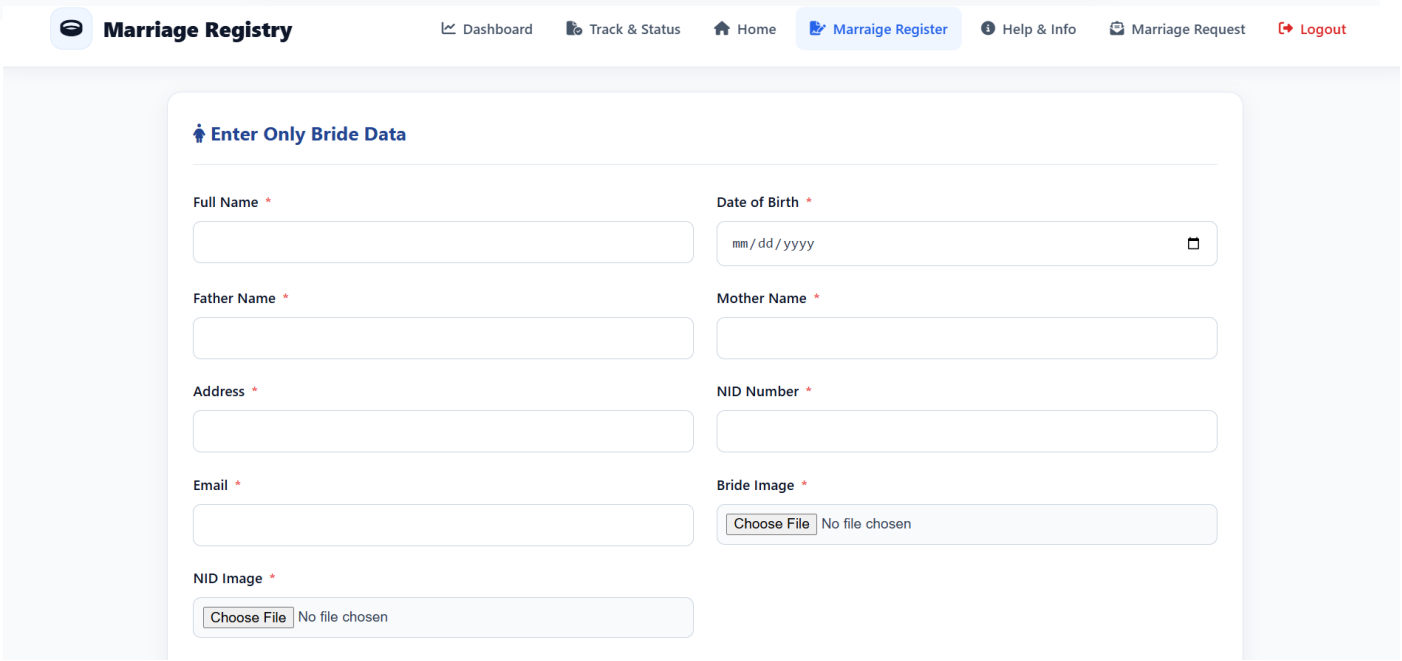

Marriage Registry
Home
Marraige Register
Help & Info
Register
Login
Register your marriage online
Apply for a marriage license, track your application, and download your certificate — all in one place.
Start Application
Check Application Status
What you can do here
Everything related to your marriage registration, handled online.

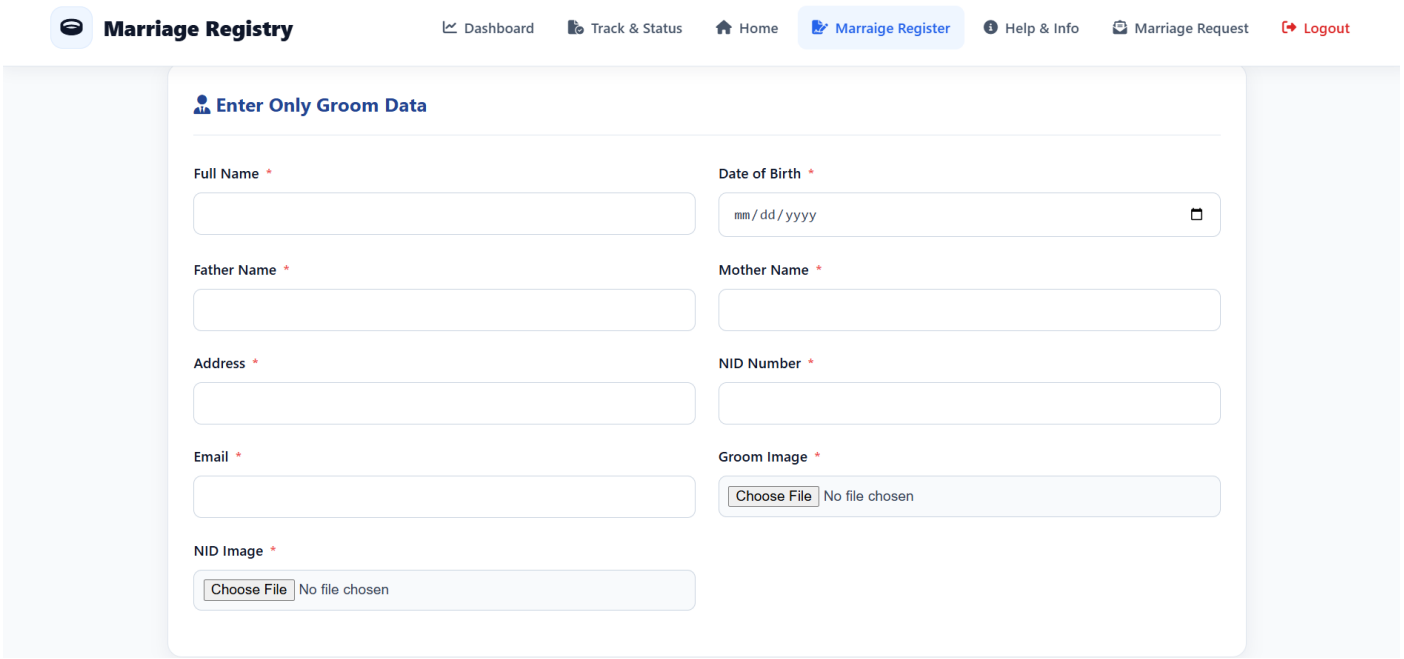

Create Account
Create your account to continue.
Username
Enter your username
Email Address
Enter your email
Password
Enter your password
Create Account

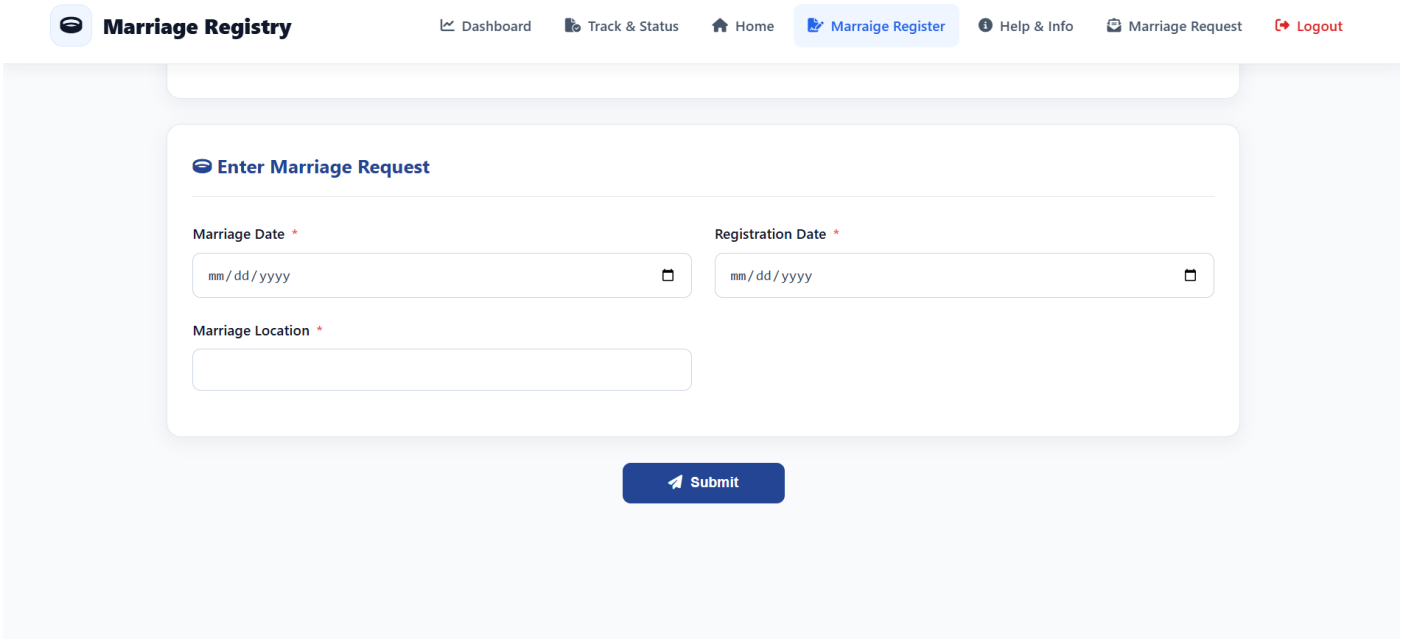

Marriage Registry
Dashboard
Track & Status
Home
Marraige Register
Help & Info
Marriage Request
Logout
ADMIN DASHBOARD
Marriage Registration Dashboard
Manage registrations, requests and marriage records.
Total Users
3
Total request
11
Pending Requests
2
Approved
6
Rejected
3
Recent Marriage Requests
Latest registration requests
View All →
Quick Actions
Frequently used options

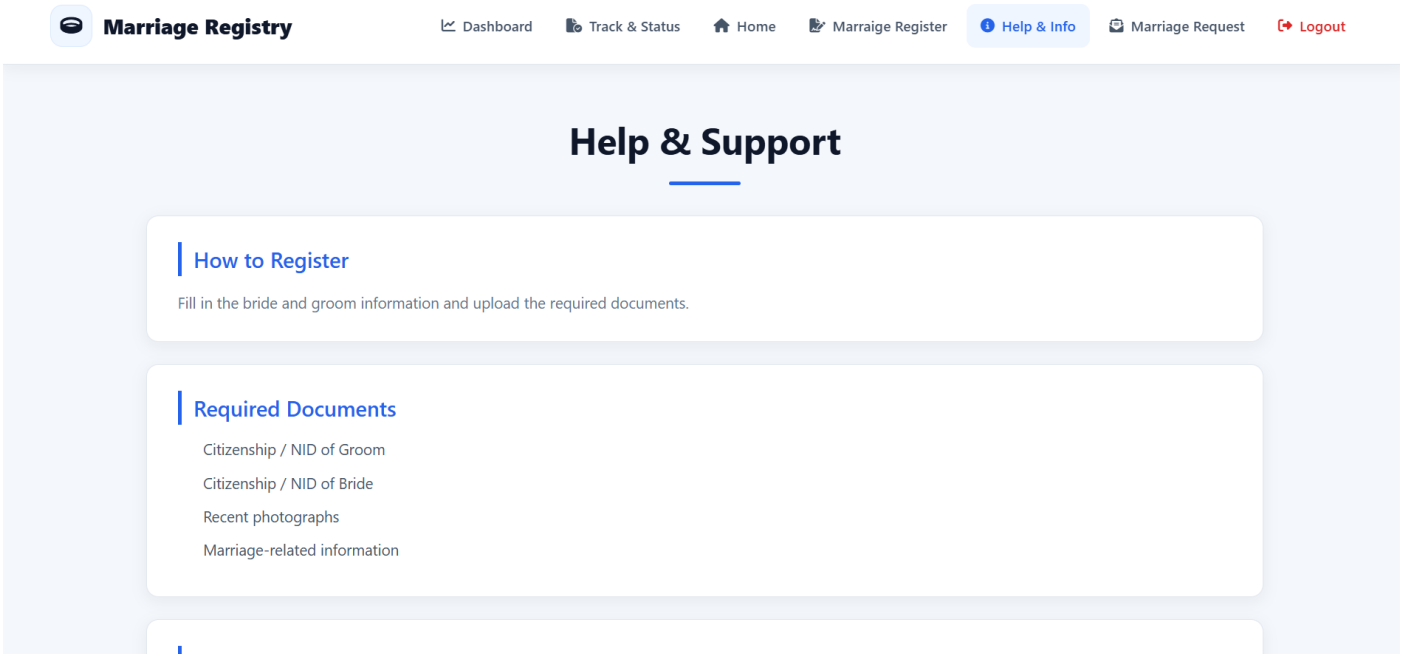

Marriage Registry
Dashboard
Track & Status
Home
Marraige Register
Help & Info
Marriage Request
Logout
10
r4yegfhdc
hjvb
ufcuigds@gmail.com
Completed
Reject
11
gharji nadshakjh5
uhrfb
uhfcb@gmail.com
Completed
Approve
Certificate
Marriage Registry
A secure and simple digital platform for marriage registration and certificate services.
Government Service Portal
Quick Links
Home
Register
Help & Info
Legal
Terms & Conditions
Privacy Policy
Data Protection
Disclaimer
Support
Contact Us
FAQ
Report Issue
Feedback
© 2026 Marriage Registration System. All Rights Reserved.
AsMin Poudel

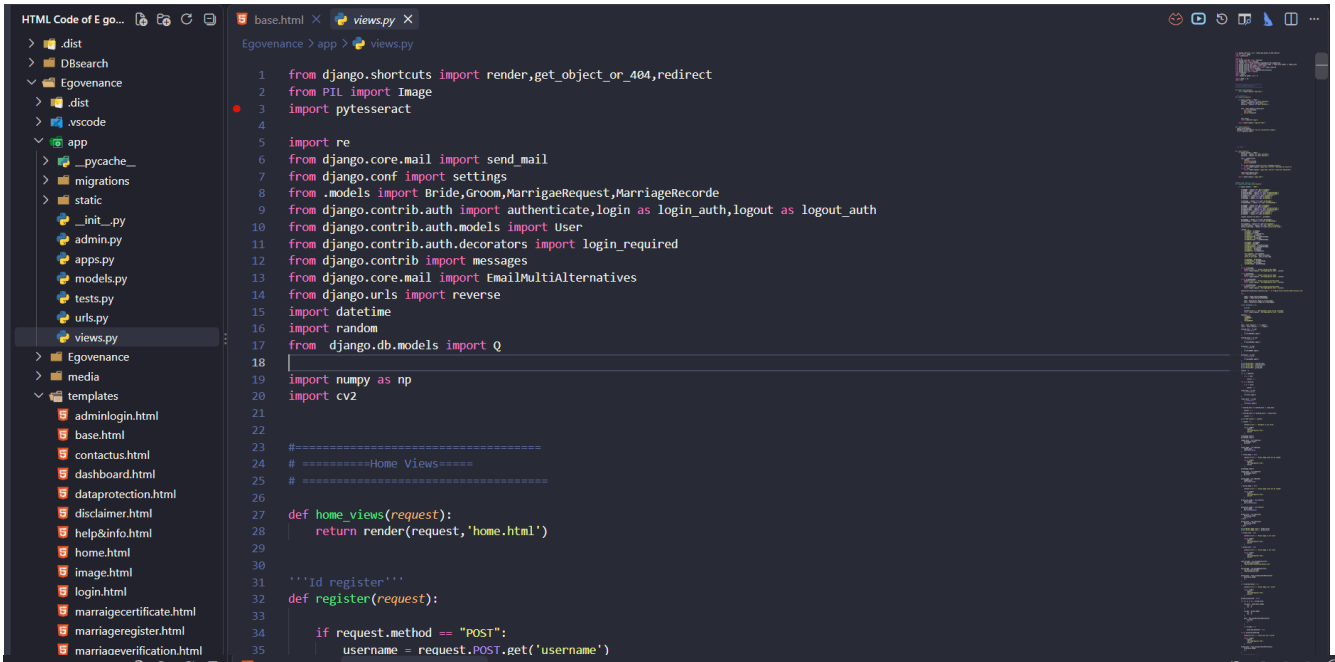

Marriage Registry
Dashboard
Track & Status
Home
Marraige Register
Help & Info
Marriage Request
Logout
OFFICIAL VERIFICATION
Marriage Verification
Verify whether a marriage has been officially registered in the marriage registration system.
Verify Marriage Record
Enter one of the details below to search for a registered marriage.
NID Number(Groom or Bride)
Enter NID number
Verify Marriage
Your information is securely processed and used only for verification.

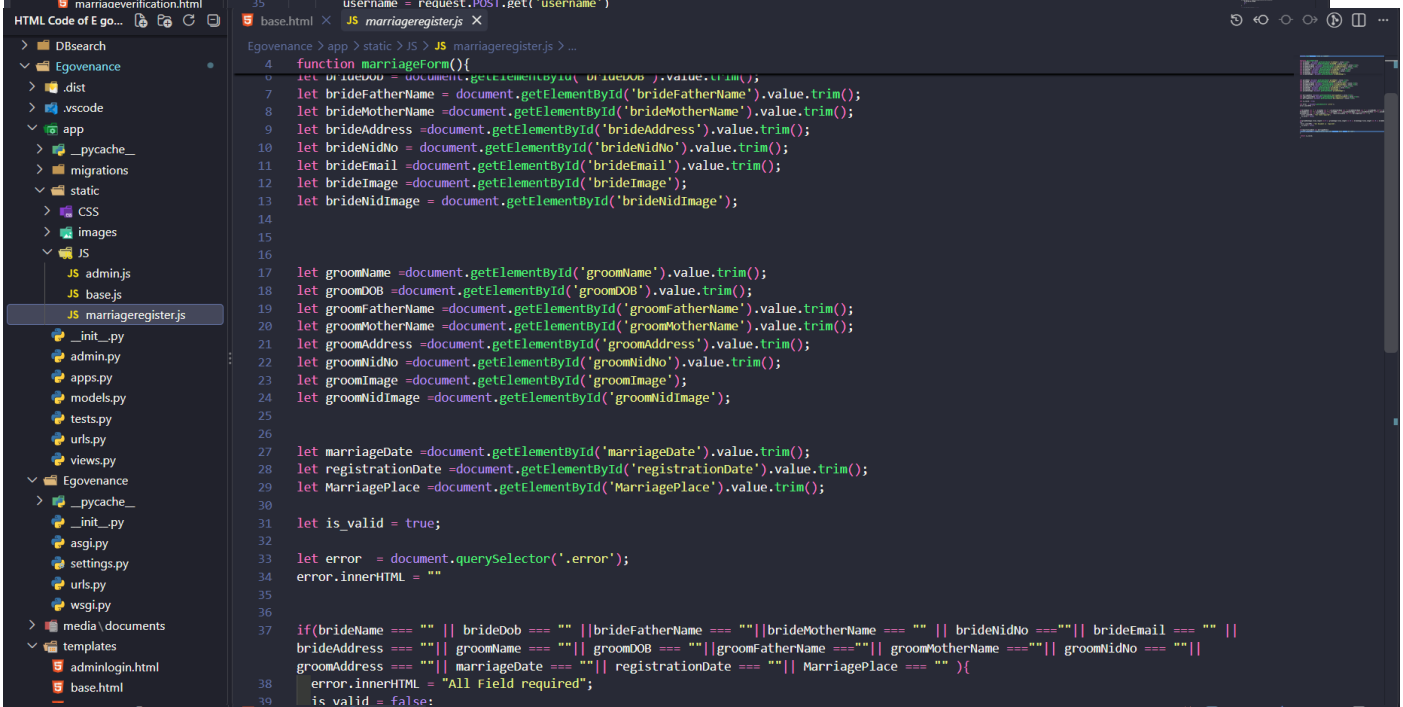

Marriage Registry
Dashboard
Track & Status
Home
Marraige Register
Help & Info
Marriage Request
Logout
Enter Only Bride Data
Full Name *
Date of Birth *
mm/dd/yyyy
Father Name *
Mother Name *
Address *
NID Number *
Email *
Bride Image *
Choose File No file chosen
NID Image *
Choose File No file chosen

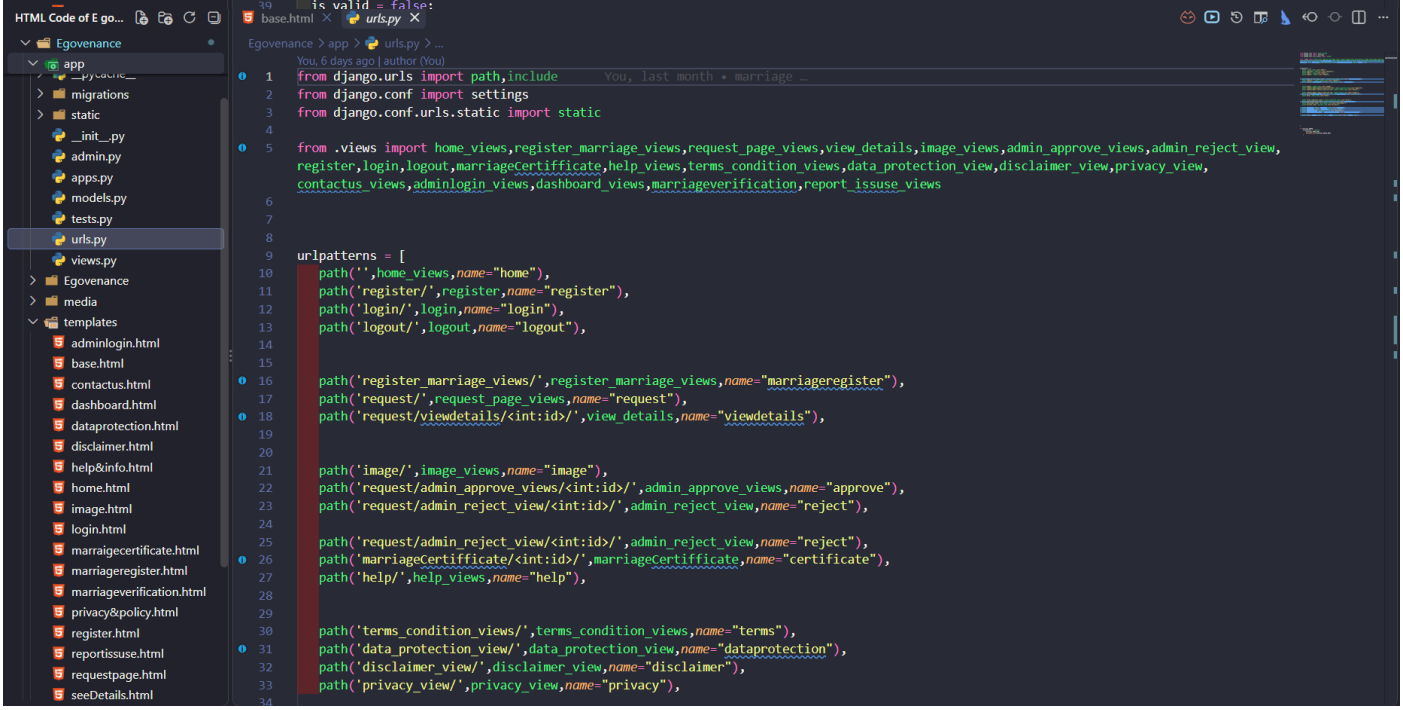

Marriage Registry
Dashboard
Track & Status
Home
Marraige Register
Help & Info
Marriage Request
Logout
Enter Only Groom Data
Full Name *
Date of Birth *
mm/dd/yyyy
Father Name *
Mother Name *
Address *
NID Number *
Email *
Groom Image *
Choose File No file chosen
NID Image *
Choose File No file chosen

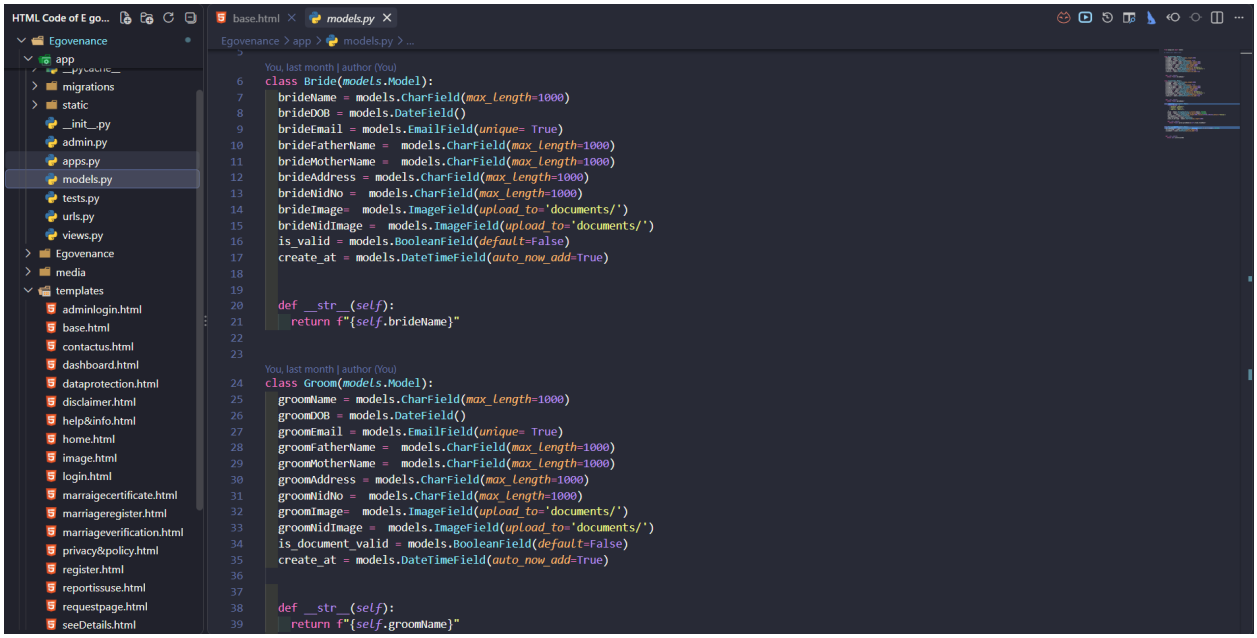

Marriage Registry
Dashboard
Track & Status
Home
Marraige Register
Help & Info
Marriage Request
Logout
Enter Marriage Request
Marriage Date *
mm/dd/yyyy
Registration Date *
mm/dd/yyyy
Marriage Location *
Submit


Marriage Registry
Dashboard
Track & Status
Home
Marraige Register
Help & Info
Marriage Request
Logout
Help & Support
How to Register
Fill in the bride and groom information and upload the required documents.
Required Documents
Citizenship / NID of Groom
Citizenship / NID of Bride
Recent photographs
Marriage-related information


HTML Code of E go...
.dist
DBsearch
Egovenance
.dist
.vscode
app
__pycache__
migrations
static
__init__.py
admin.py
apps.py
models.py
tests.py
urls.py
views.py
Egovenance
media
templates
adminlogin.html
base.html
contactus.html
dashboard.html
dataprotection.html
disclaimer.html
help&info.html
home.html
image.html
login.html
marraigecertificate.html
marriageregister.html
marriageverification.html
base.html
views.py
Egovenance > app > views.py
from django.shortcuts import render,get_object_or_404,redirect
from PIL import Image
import pytesseract
import re
from django.core.mail import send_mail
from django.conf import settings
from .models import Bride,Groom,MarrigaeRequest,MarriageRecorde
from django.contrib.auth import authenticate,login as login_auth,logout as logout_auth
from django.contrib.auth.models import User
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from django.core.mail import EmailMultiAlternatives
from django.urls import reverse
import datetime
import random
from django.db.models import Q
import numpy as np
import cv2
#==================================
=========Home Views=====
==================================
def home_views(request):
return render(request,'home.html')
'''Id register'''
def register(request):
if request.method == "POST":
username = request.POST.get('username')


HTML Code of E go...
DBsearch
Egovenance
.dist
.vscode
app
__pycache__
migrations
static
CSS
images
JS
admin.js
base.js
marriageregister.js
__init__.py
admin.py
apps.py
models.py
tests.py
urls.py
views.py
Egovenance
__pycache__
__init__.py
asgi.py
settings.py
urls.py
wsgi.py
media\documents
templates
adminlogin.html
base.html
base.html
marriageregister.js
Egovenance > app > static > JS > marriageregister.js > ...
function marriageForm(){
let brideDob = document.getElementById('brideDob').value.trim();
let brideFatherName = document.getElementById('brideFatherName').value.trim();
let brideMotherName =document.getElementById('brideMotherName').value.trim();
let brideAddress =document.getElementById('brideAddress').value.trim();
let brideNidNo = document.getElementById('brideNidNo').value.trim();
let brideEmail =document.getElementById('brideEmail').value.trim();
let brideImage =document.getElementById('brideImage');
let brideNidImage = document.getElementById('brideNidImage');
let groomName =document.getElementById('groomName').value.trim();
let groomDOB =document.getElementById('groomDOB').value.trim();
let groomFatherName =document.getElementById('groomFatherName').value.trim();
let groomMotherName =document.getElementById('groomMotherName').value.trim();
let groomAddress =document.getElementById('groomAddress').value.trim();
let groomNidNo =document.getElementById('groomNidNo').value.trim();
let groomImage =document.getElementById('groomImage');
let groomNidImage =document.getElementById('groomNidImage');
let marriageDate =document.getElementById('marriageDate').value.trim();
let registrationDate =document.getElementById('registrationDate').value.trim();
let MarriagePlace =document.getElementById('MarriagePlace').value.trim();
let is_valid = true;
let error = document.querySelector('.error');
error.innerHTML = ""
if(brideName === "" || brideDob === "" ||brideFatherName === ""||brideMotherName === "" || brideNidNo ===""|| brideEmail === "" ||
brideAddress === ""|| groomName === ""|| groomDOB === ""||groomFatherName ===""|| groomMotherName ===""|| groomNidNo === ""||
groomAddress === ""|| marriageDate === ""|| registrationDate === ""|| MarriagePlace === ""){
error.innerHTML = "All Field required";
is_valid = false;


HTML Code of E go...
Egovenance
app
migrations
static
__init__.py
admin.py
apps.py
models.py
tests.py
urls.py
views.py
Egovenance
media
templates
adminlogin.html
base.html
contactus.html
dashboard.html
dataprotection.html
disclaimer.html
help&info.html
home.html
image.html
login.html
marraigecertificate.html
marriageregister.html
marriageverification.html
privacy&policy.html
register.html
reportissue.html
requestpage.html
seeDetails.html
base.html
urls.py
Egovenance > app > urls.py > ...
You, 6 days ago | author (You)
from django.urls import path,include
You, last month • marriage ...
from django.conf import settings
from django.conf.urls.static import static
from .views import home_views,register_marriage_views,request_page_views,view_details,image_views,admin_approve_views,admin_reject_view,
register,login,logout,marriageCertitificate,help_views,terms_condition_views,data_protection_view,disclaimer_view,privacy_view,
contactus_views,adminlogin_views,dashboard_views,marriageverification,report_issuse_views
urlpatterns = [
path('',home_views,name="home"),
path('register/',register,name="register"),
path('login/',login,name="login"),
path('logout/',logout,name="logout"),
path('register_marriage_views/',register_marriage_views,name="marriageregister"),
path('request/',request_page_views,name="request"),
path('request/viewdetails/<int:id>/',view_details,name="viewdetails"),
path('image/',image_views,name="image"),
path('request/admin_approve_views/<int:id>/',admin_approve_views,name="approve"),
path('request/admin_reject_view/<int:id>/',admin_reject_view,name="reject"),
path('request/admin_reject_view/<int:id>/',admin_reject_view,name="reject"),
path('marriageCertitificate/<int:id>/',marriageCertitificate,name="certificate"),
path('help/',help_views,name="help"),
path('terms_condition_views/',terms_condition_views,name="terms"),
path('data_protection_view/',data_protection_view,name="dataprotection"),
path('disclaimer_view/',disclaimer_view,name="disclaimer"),
path('privacy_view/',privacy_view,name="privacy"),

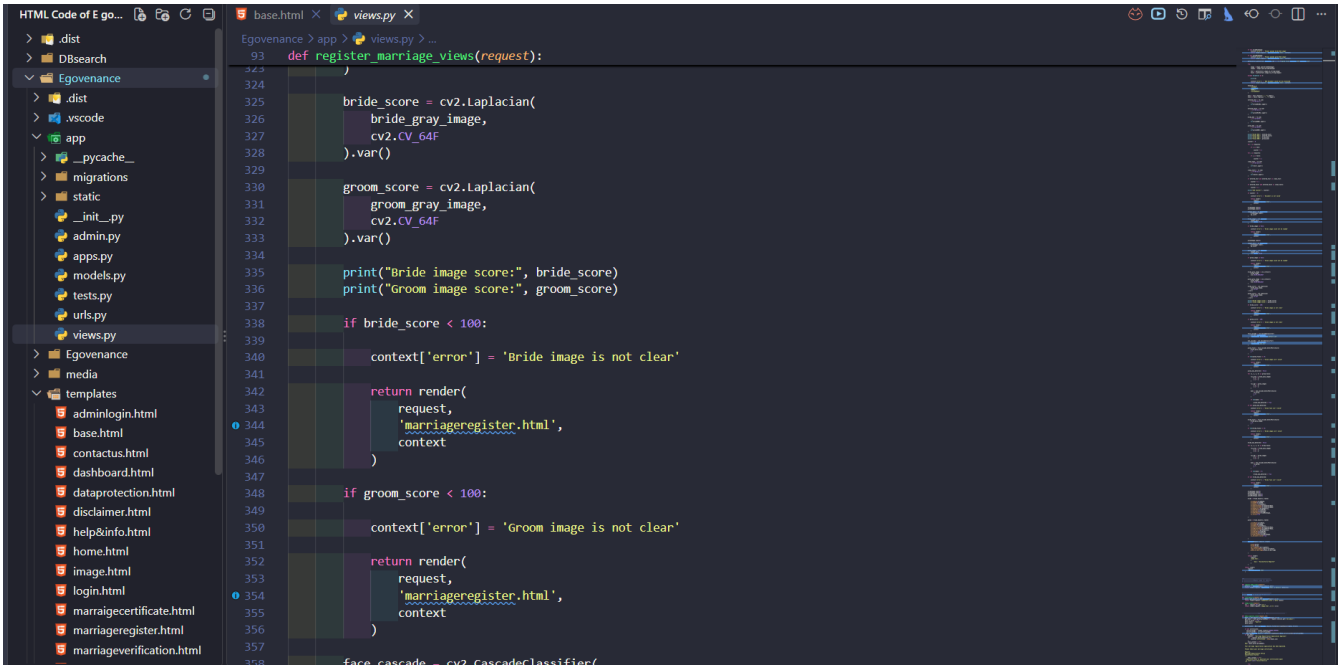


```
You, last month | author (You)
class Bride(models.Model):
  brideName = models.CharField(max_length=1000)
  brideDOB = models.DateField()
  brideEmail = models.EmailField(unique= True)
  brideFatherName =  models.CharField(max_length=1000)
  brideMotherName =  models.CharField(max_length=1000)
  brideAddress = models.CharField(max_length=1000)
  brideNidNo =  models.CharField(max_length=1000)
  brideImage=  models.ImageField(upload_to='documents/')
  brideNidImage =  models.ImageField(upload_to='documents/')
  is_valid = models.BooleanField(default=False)
  create_at = models.DateTimeField(auto_now_add=True)


  def __str__(self):
    return f"{self.brideName}"


You, last month | author (You)
class Groom(models.Model):
  groomName = models.CharField(max_length=1000)
  groomDOB = models.DateField()
  groomEmail = models.EmailField(unique= True)
  groomFatherName =  models.CharField(max_length=1000)
  groomMotherName =  models.CharField(max_length=1000)
  groomAddress = models.CharField(max_length=1000)
  groomNidNo =  models.CharField(max_length=1000)
  groomImage=  models.ImageField(upload_to='documents/')
  groomNidImage =  models.ImageField(upload_to='documents/')
  is_document_valid = models.BooleanField(default=False)
  create_at = models.DateTimeField(auto_now_add=True)


  def __str__(self):
    return f"{self.groomName}"
```



```
def register_marriage_views(request):
        )

        bride_score = cv2.Laplacian(
            bride_gray_image,
            cv2.CV_64F
        ).var()

        groom_score = cv2.Laplacian(
            groom_gray_image,
            cv2.CV_64F
        ).var()

        print("Bride image score:", bride_score)
        print("Groom image score:", groom_score)

        if bride_score < 100:

            context['error'] = 'Bride image is not clear'

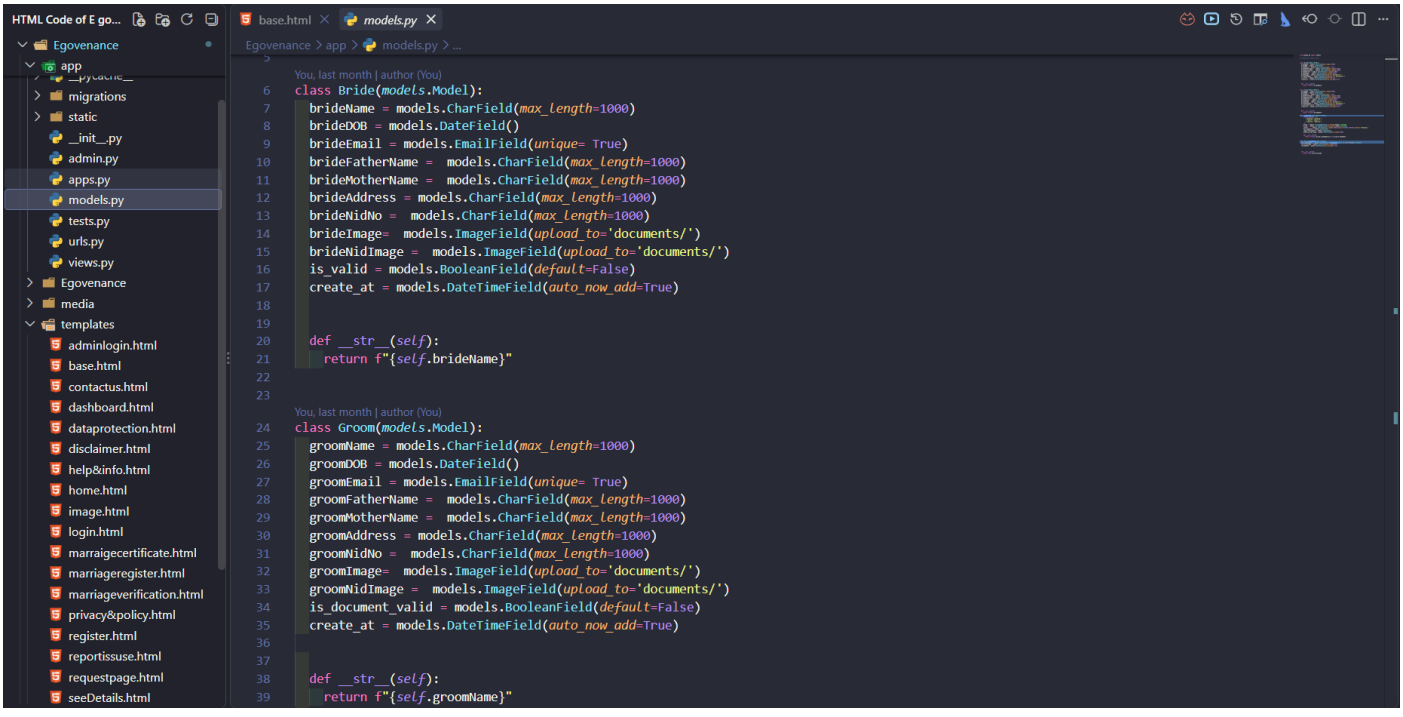
            return render(
                request,
                'marriageregister.html',
                context
            )

        if groom_score < 100:

            context['error'] = 'Groom image is not clear'

            return render(
                request,
                'marriageregister.html',
                context
            )

        face_cascade = cv2.CascadeClassifier(
```



```
You, last month | author (You)
class Bride(models.Model):
  brideName = models.CharField(max_length=1000)
  brideDOB = models.DateField()
  brideEmail = models.EmailField(unique= True)
  brideFatherName =  models.CharField(max_length=1000)
  brideMotherName =  models.CharField(max_length=1000)
  brideAddress = models.CharField(max_length=1000)
  brideNidNo =  models.CharField(max_length=1000)
  brideImage=  models.ImageField(upload_to='documents/')
  brideNidImage =  models.ImageField(upload_to='documents/')
  is_valid = models.BooleanField(default=False)
  create_at = models.DateTimeField(auto_now_add=True)


  def __str__(self):
    return f"{self.brideName}"


You, last month | author (You)
class Groom(models.Model):
  groomName = models.CharField(max_length=1000)
  groomDOB = models.DateField()
  groomEmail = models.EmailField(unique= True)
  groomFatherName =  models.CharField(max_length=1000)
  groomMotherName =  models.CharField(max_length=1000)
  groomAddress = models.CharField(max_length=1000)
  groomNidNo =  models.CharField(max_length=1000)
  groomImage=  models.ImageField(upload_to='documents/')
  groomNidImage =  models.ImageField(upload_to='documents/')
  is_document_valid = models.BooleanField(default=False)
  create_at = models.DateTimeField(auto_now_add=True)


  def __str__(self):
    return f"{self.groomName}"
```